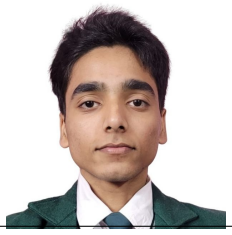
Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

DNA (Destructured Neural Agent): Evolutionary Autonomous Robotic Navigation Simulation Using Genetic Algorithms in C++

Team Name: *Team #*	*Evowalk* *T051*
Team member 1 (Team Lead) *Mahar, Tanishka -2510011047* *mahartanishka068@gmail.com*	
Team member 2 *Juglan, Alankrit -2510350082* *alankritjuglan@gmail.com*	
Team member 3 *Khati, Mansha - 2510380049* *manshaakhatiii@gmail.com*	
Team member 4 *Vinamra - 2510012765* *victorvc3018@gmail.com*	

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

Autonomous robotic navigation often fails in unknown or dynamic environments when it is constrained by rigid, pre-programmed rules. The DNA (Destructured Neural Agent) project tackles this by creating a software-based evolutionary robot simulation in C++. Instead of manually coding each navigational situation, the virtual robot uses a Genetic Algorithm (GA) to find, test and refine its own movement paths over many generations.

We have now successfully implemented and verified the core evolutionary engine in C++. This includes population generation, fitness scoring, proportional selection with elitism, single-point crossover, and mutation.

Currently, we are preparing to combine this engine with Python for the real-time 2D graphical visualization of obstacle avoidance and path convergence. The project demonstrates a lean example of adaptive evolutionary behavior in computational robotics based on fundamental Data Structures and modular programming.

Updated Project Approach and Architecture (2 pts)

Our approach centers on a modular desktop simulation built around a high-performance C++ algorithmic core, coupled with Python for 2D graphical visualization.
The system architecture consists of three integrated layers:

1. **Environment Layer:** Manages the virtual 2D arena, encompassing boundary coordinates, user-defined obstacle bounding boxes, and target locations. It tracks spatial limits and supplies environmental feedback to the agent

2. **Genetic Algorithm Engine:** The core of the optimization. A population of candidate movement strategies (chromosomes) is maintained. Each chromosome is a chain of directional movement steps.
   The evolutionary pipeline consists of:
   - *Fitness Evaluation:* Evaluates paths based on Euclidean distance to target, total steps taken, and collision penalties.
   - *Selection:* Employs roulette-wheel proportional selection with elitism to preserve the best performers while maintaining the genetic variance of the larger population
   - *Genetic Operators:* Recombination of parent strategies via single point crossover, stochastic mutation (rate 5%) to introduce new directional steps.

3. **Visualization Layer:** It draws the simulation arena, the geometry of the obstacles and real-time or robot trajectories at the end of the generation.

As the system is executed as a self-contained desktop application, inter-module communication is synchronous and execute directly without external network protocols. Standard C++ libraries (`<iostream>`, `<cstdlib>`, `<cmath>`, `<ctime>`, `<cstring>`) are used for the core algorithm logic, keeping the simulation fast, lightweight, and Platform cross.

Tasks Completed (7 pts)

Task Completed	Team Member
System architecture, workflow design, and overall project planning: System specifications, high-performance C++ algorithm core interfaces, class hierarchies, and 2D visualization using Python.	Vinamra (Lead)
GA core prototype implementation and algorithmic engine development: Created and tested working evolutionary engine in C++ using 150 individual population with seeding, evaluation, crossover, and mutation.	Alankrit Juglan (Developer)
Fitness scoring design, selection distribution, and mutation logic: Fitness function calculations were done through Euclidean distance and obstacle cost, roulette wheel selection distribution was adjusted, and mutation probability was set to 0.05.	Mansha Khati (Algorithm)
Prototype testing, validation tracking, and milestone documentation: Performed comprehensive testing of all units in the areas of initialization, integrity of crossover operation, convergence stability and milestone achievement measurement.	Tanishka Mahar (Testing/Docs)

Challenges/Roadblocks (7 pts)

During Phase-I development, our team worked through several algorithmic problems and identified the main technical hurdles still ahead of integration:

1. **Premature Convergence and Lack of Diversity:**
   Early on, high-fitness candidates dominated the selection process, and the population got stuck in local optima. We fixed this by adjusting the share calculation (`share_calc`) so every individual gets a minimum reproductive share, and by lowering the mutation rate to 5%.
2. **Mapping 1D Chromosomes to 2D Spatial Movement:**
   The prototype scored paths using simple discrete characters, but a robot needs 2D spatial vectors and collision detection to move around. To handle this, we are writing direction-based translation functions that turn each chromosome index into a coordinate vector (dx, dy) for Python.
3. **Simulation Performance vs. Rendering:**
   Drawing every frame for all 150 members adds a lot of visual overhead and slows things down. Our plan is to separate the physics and evolution loop from the rendering loop: C++ will run the generations in headless mode at full CPU speed, and Python will only be updated with the best-performing trajectories.
4. **Dynamic Memory Management:**
   The prototype used static arrays, which was fine for initial testing. To support different population sizes and paths of any length, we are moving to dynamic memory management with STL containers (`std::vector`).

Tasks Pending (7 pts)

Task Pending	**Team Member (to complete the task)**
Python 2D simulation environment and graphical arena rendering pipeline	Alankrit Juglan (Developer)
Robot movement translation (1D to 2D displacement vectors) and obstacle collision physics	Mansha Khati (Algorithm)
Generation-wise trajectory visualization and live metrics display	Vinamra (Team Lead) & Alankrit Juglan
Dynamic STL container refactoring (`std::vector`), integration testing, and documentation	Tanishka Mahar (Testing/Docs) & Vinamra

Project Outcome/Deliverables (2 pts)

The primary outcome is an interactive, lightweight desktop application that visually demonstrates autonomous obstacle avoidance and path planning through genetic evolution.

Key Deliverables:

1. **Standalone Genetic Algorithm Engine:** A robust, reusable C++ engine handling population seeding, fitness evaluation, roulette-wheel selection, crossover, and mutation.
2. **Python 2D Simulation Environment:** A graphical interface displaying the robot, arena boundaries, customizable obstacles, and target objectives.
3. **Adaptive Trajectory Visualization:** Clear generation-by-generation rendering of candidate paths, highlighting the best-performing trajectory and path refinements over time.
4. **Performance Metrics Display:** Real-time on-screen reporting of generation counts, best fitness scores, convergence rates, and success status.
5. **Well-Structured Codebase & Documentation:** Fully commented C++ code demonstrating core Data Structures and algorithmic concepts, accompanied by comprehensive build instructions and project reports.

Progress Overview (2 pts)

Generally speaking, our current work is 50% finished and on schedule to complete our project goals for our final evaluation.

- **Ahead of Schedule:** The core logic behind the Genetic Algorithm, including its random initial population of 150 individuals, fitness calculations, proportional selection with elitism, crossover, and mutation have all been implemented and tested early via our prototype in C++ (`pbl.cpp`). We have achieved algorithm convergence.
- **On Schedule:** System architectural specifications, team task distribution, object-oriented data structures and implementation, and testing of the core algorithm in C++ have all been completed.
- **Behind Schedule:** Linking the chromosome outputs of our algorithms with 2D collision detection and graphically rendering them in Python will be the next key objective, although this was put on hold for the purpose of implementing the selection and convergence mechanism of the core algorithm. Our graphical integration in Python is now on schedule.

Codebase Information (2 pts)

Repository Link: `https://github.com/Evowalk-T051/DNA-Destructured-Neural-Agent`
Active Branch: `main` (development branch: `feature/python-visualization`)
Important Commits:

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Population Initialization Test	**Pass**	Checked that 150 individuals were generated randomly within the defined sample set.
Fitness Function Verification	**Pass**	Confirmed that candidate strings are scored correctly against the target.
Selection Distribution Test	**Pass**	Checked that fitter candidates get a proportionally bigger share of reproduction.
Single-Point Crossover Test	**Pass**	Made sure offspring pick up valid segments from both parents with nothing lost.
Mutation Rate Boundary Test	**Pass**	Confirmed that characters mutate randomly and stay within the 5% probability limit.
Convergence & Termination Test	**Pass**	Confirmed the population fully converges on the target over successive generations and the loop always terminates.

Deliverables Progress (2 pts)

1. **Main Genetic Algorithm Engine:** *Done* – Fully functional prototype implemented in C++ with selection, crossover and mutation.
2. **System Architecture and Pipeline Design:** *Done* – Diagrams and interfaces for workflow and procedural flow done.
3. **Python Graphics Visualization Pipeline:** *In Progress* – Preparing and designing the 2D graphical visualization interface using Python.
4. **Robot Movement and Collision Physics:** *In Progress* – Mapping the chromosome genes to 2D vector movement and obstacles boundaries.
5. **Visualizing Trajectories per Generation:** *In Progress* – Implementation of real time trajectory visualization and highlighting the best trajectory.
6. **Final Integration and Documentation:** *In Progress* – Commenting on code, milestone report and demonstration material.